

10PEARLS PAKISTAN

Software Quality Assurance Internship Program

Assignment 4 – API Testing

Week 4 Internship Assignment

Submitted By:	Muqadas Shahid
Organization:	10Pearls Pakistan
Assignment:	Assignment 4 – API Testing (Week 4)
Tools Used:	Postman, Apache JMeter 5.6.3
API Under Test:	JSONPlaceholder REST API
Base URL:	https://jsonplaceholder.typicode.com
Submission Date:	August 16, 2026

Table of Contents

1. Introduction	3
2. Objectives.....	3
3. Tools and Technologies.....	3
4. API Endpoints Tested	3
5. Postman Functional Testing.....	4
6. Automated Test Assertions.....	7
7. JMeter Performance Testing.....	7
8. Performance Results and Analysis	10
9. Key Findings	12
10. Conclusion.....	12
11. Learning Outcomes	12
12. GitHub Repository.....	13

1. Introduction

API (Application Programming Interface) testing is a type of software testing that verifies whether an API meets expectations for functionality, reliability, and performance. It involves sending requests directly to API endpoints and validating the returned status codes, response structure, and data, without relying on a graphical user interface.

In software quality assurance, API testing is important because APIs form the communication layer between different systems and services. Validating APIs early in the development cycle helps catch defects before they affect the user interface, supports automated regression testing, and improves overall confidence in system integration.

Functional API testing verifies that each endpoint behaves correctly according to its specification — checking status codes, JSON structure, and returned data for individual requests. Performance testing, on the other hand, evaluates how the API behaves when multiple requests are sent repeatedly or concurrently, measuring response time, throughput, and error rate under load. This assignment covered both types of testing using Postman and Apache JMeter respectively.

2. Objectives

- Validate REST API functionality for the JSONPlaceholder API.
- Test CRUD (Create, Read, Update, Delete) operations on the /posts resource.
- Validate HTTP status codes and response data for each request.
- Verify the structure of JSON responses returned by the API.
- Execute automated API assertions using Postman test scripts.
- Evaluate API performance under multiple simulated users using Apache JMeter.
- Analyze response time, throughput, and error rate under repeated requests.

3. Tools and Technologies

Tool	Purpose
Postman	Functional API testing and automated assertions
Apache JMeter 5.6.3	Performance and load testing
JSONPlaceholder	REST API used for testing (https://jsonplaceholder.typicode.com)

4. API Endpoints Tested

Test ID	Method	API Request	Endpoint	Purpose
TC-01	GET	All Posts	/posts	Retrieve all posts

Test ID	Method	API Request	Endpoint	Purpose
TC-02	GET	Single Post	/posts/1	Retrieve a single post
TC-03	POST	Create Post	/posts	Create a new post
TC-04	PUT	Update Post	/posts/1	Update an existing post
TC-05	DELETE	Delete Post	/posts/1	Delete a post

5. Postman Functional Testing

A Postman Collection named "Assignment 4 - API Testing" was created to organize all requests used for functional testing (Figure 1). The collection contains five requests covering all CRUD operations on the /posts resource: GET - All Posts, GET - Single Post, POST - Create Post, PUT - Update Post, and DELETE - Delete Post.

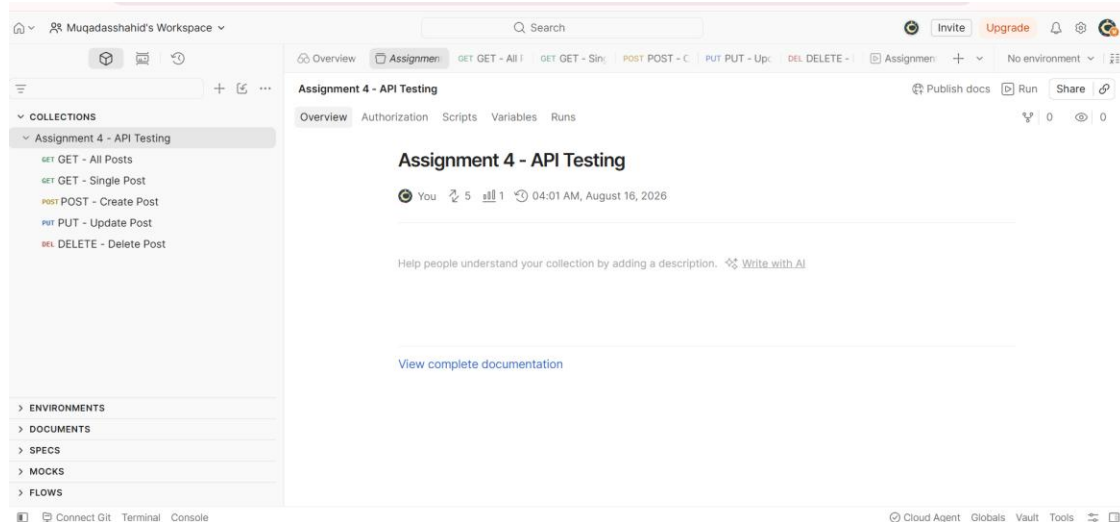


Figure 1: Postman API Testing Collection – "Assignment 4 - API Testing".

For each request, an automated Post-response test script was written using Postman's `pm.test()` API to validate the response. The GET - All Posts request, for example, returned the full list of posts along with its JSON body, as shown in Figure 2.

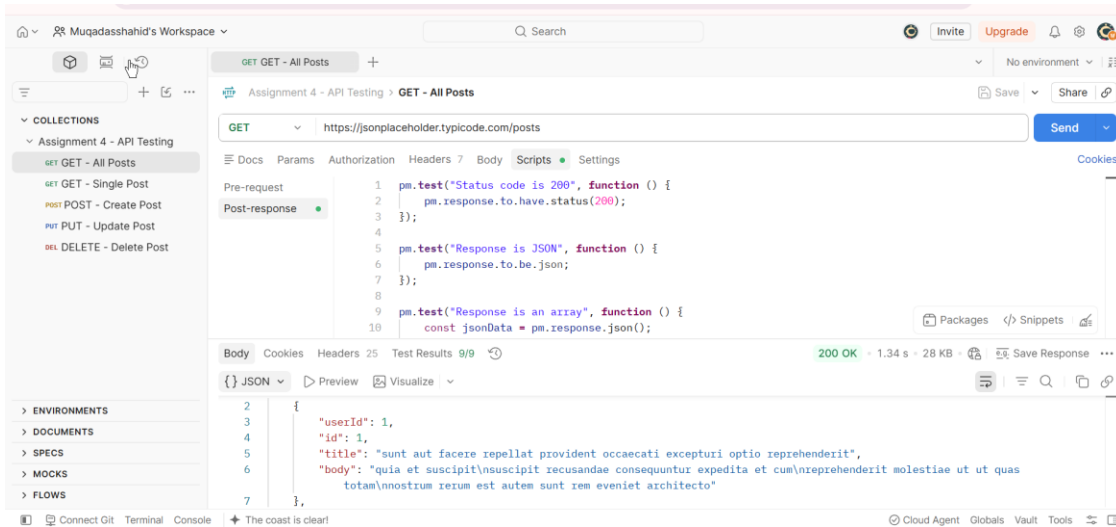


Figure 2: GET All Posts API request and JSON response (HTTP 200 OK).

The automated Postman test scripts for this request validate the status code, response format, structure, required fields, and response time, as shown in Figure 3.

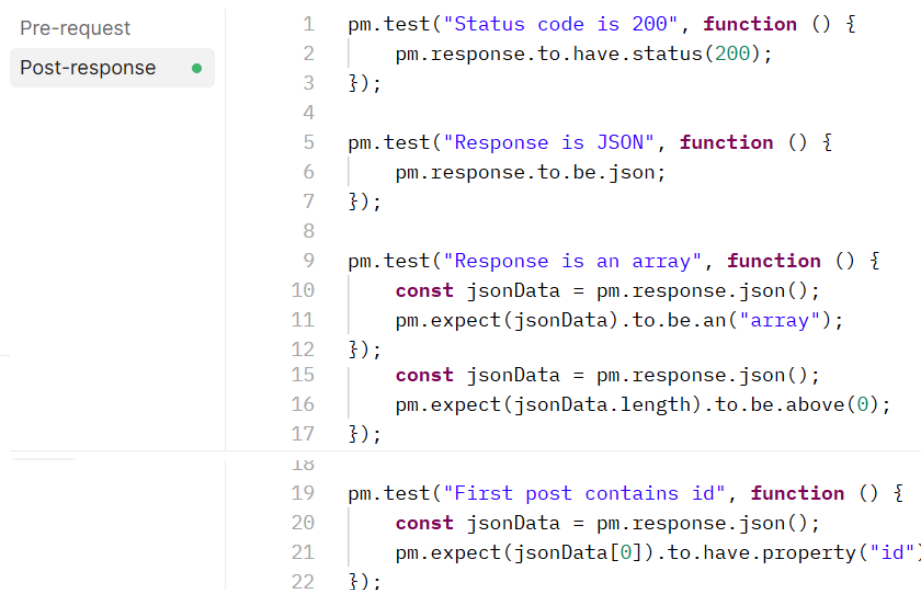


Figure 3: Automated Postman post-response test scripts for the GET All Posts request.

Running these tests in Postman produced 9 out of 9 tests passed for the GET All Posts request, confirming that the API response met all defined functional validation criteria (Figure 4).

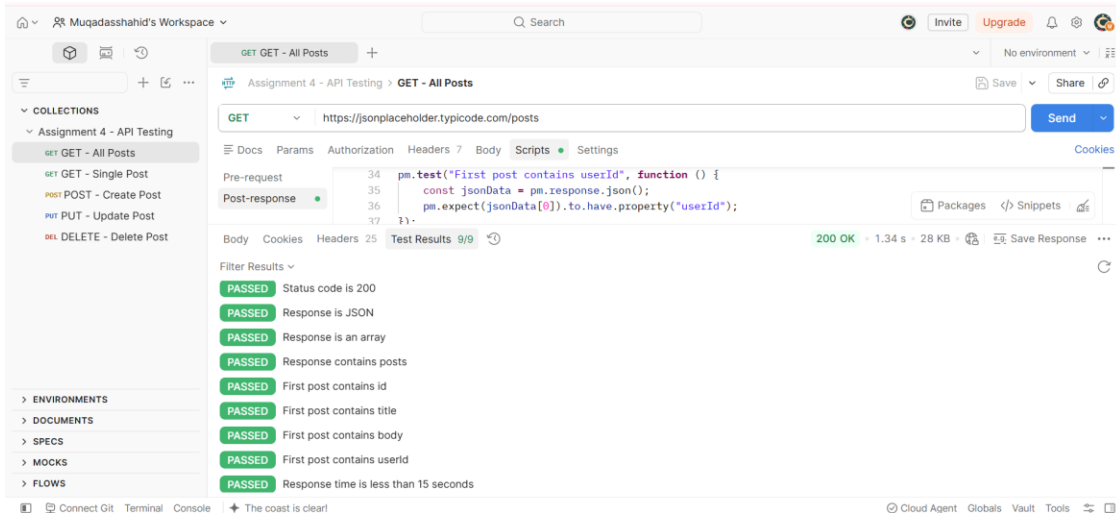


Figure 4: Postman Test Results – 9/9 tests passed for the GET All Posts request (HTTP 200 OK).

All five requests in the collection were then executed together using the Postman Collection Runner. The consolidated run results are shown in Figure 5.

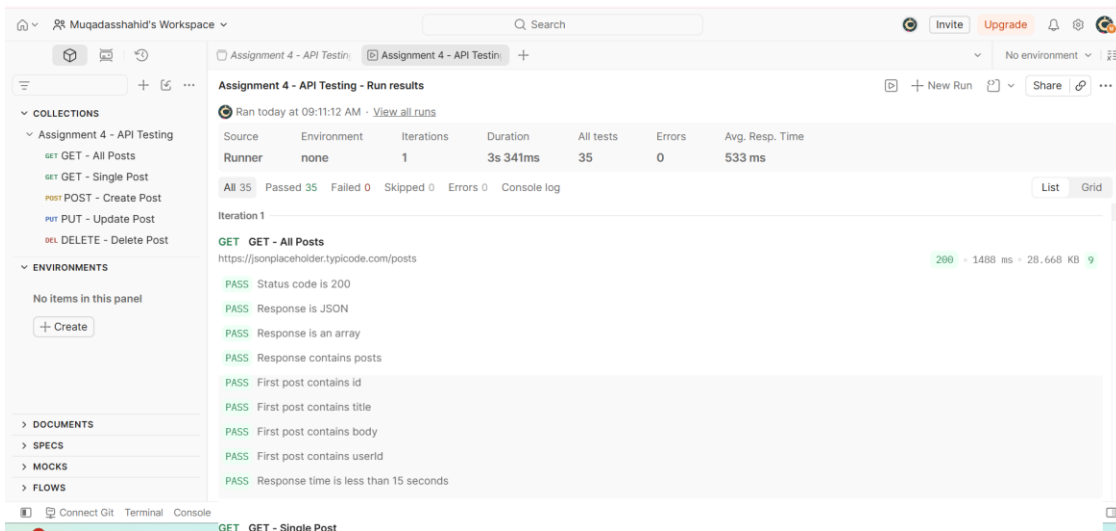


Figure 5: Postman Collection Runner results – all 35 automated tests passed across the five CRUD requests.

Metric	Result
Total Tests	35
Passed	35
Failed	0
Skipped	0
Errors	0
Run Duration	3 seconds 341 milliseconds

Metric	Result
Average Response Time	533 ms

The Collection Runner results confirm that all automated assertions across the five CRUD requests (GET - All Posts, GET - Single Post, POST - Create Post, PUT - Update Post, and DELETE - Delete Post) passed successfully, with zero failures and zero errors.

6. Automated Test Assertions

The table below lists the automated test assertions implemented for the GET - All Posts request, corresponding to the 9/9 passed result shown in Figure 4.

Test Case	Validation	Result
TC-01.1	Status code is 200	PASSED
TC-01.2	Response is JSON	PASSED
TC-01.3	Response is an array	PASSED
TC-01.4	Response contains posts	PASSED
TC-01.5	First post contains id	PASSED
TC-01.6	First post contains title	PASSED
TC-01.7	First post contains body	PASSED
TC-01.8	First post contains userId	PASSED
TC-01.9	Response time is within the defined limit (< 15 seconds)	PASSED

7. JMeter Performance Testing

Apache JMeter 5.6.3 was used to evaluate API performance under repeated requests from multiple simulated users. The test plan, named "API Load Testing - JSONPlaceholder", included HTTP Request samplers for all five CRUD operations (GET - All Posts, GET - Single Post, POST - Create Post, PUT - Update Post, and DELETE - Delete Post) together with an HTTP Header Manager configured to send Content-Type: application/json (Figure 7).

The Thread Group was configured as follows:

- Number of Threads (Users): 10
- Ramp-Up Period: 5 seconds
- Loop Count: 5
- Action on Sampler Error: Continue

This configuration simulates 10 virtual users, each sending the full set of API requests 5 times, for a total of 50 samples per request type (10 users × 5 loops).

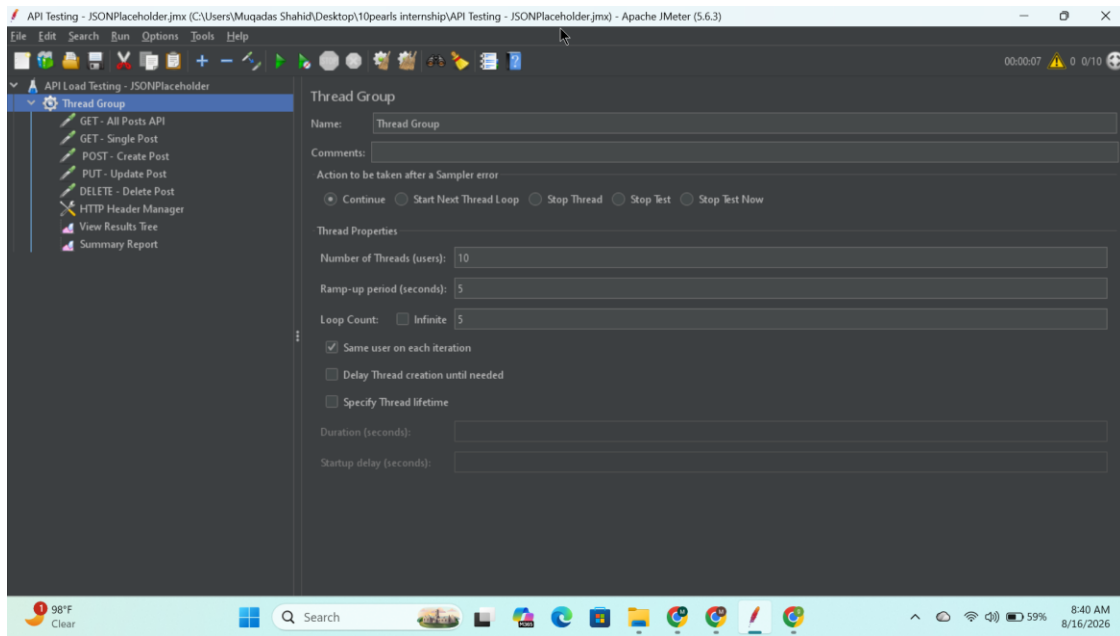


Figure 6: JMeter Thread Group configuration – 10 users, 5-second ramp-up, 5 loop iterations.

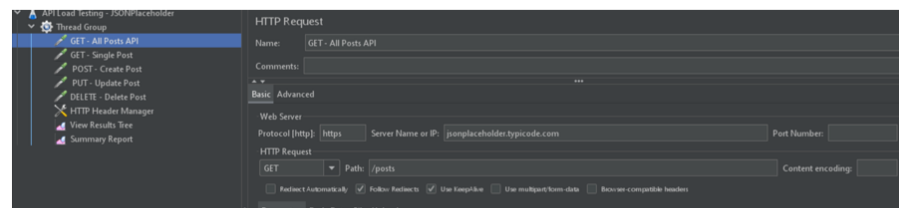
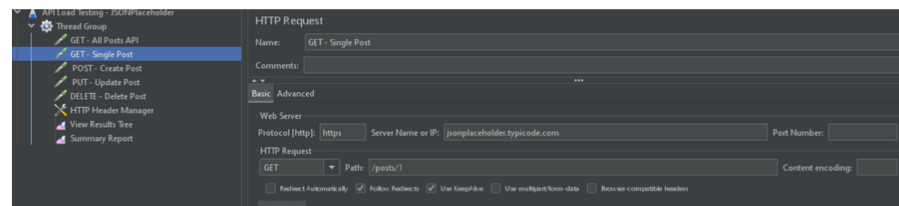
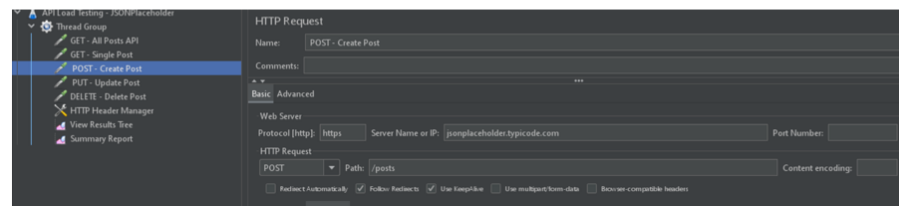
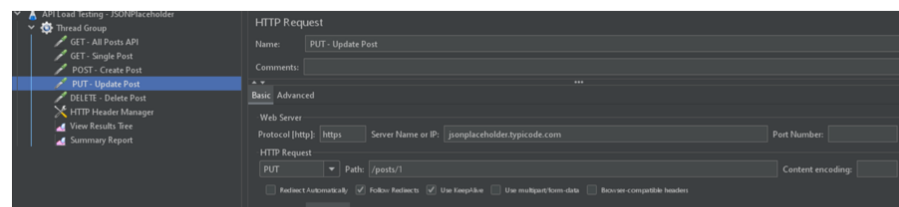
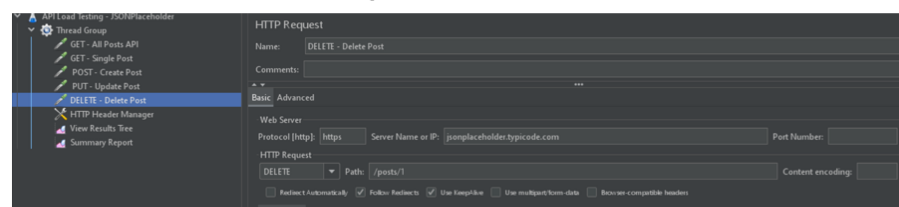
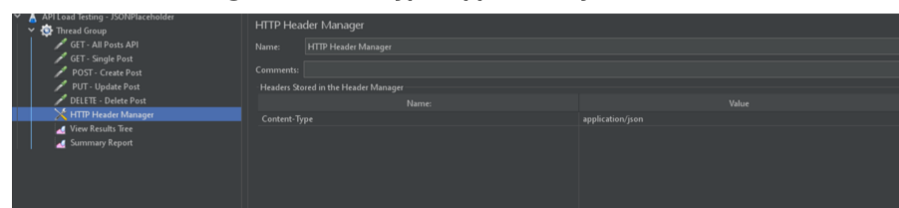
GET - All Posts API (GET /posts)**GET - Single Post (GET /posts/1)****POST - Create Post (POST /posts)****PUT - Update Post (PUT /posts/1)****DELETE - Delete Post (DELETE /posts/1)****HTTP Header Manager (Content-Type: application/json)**

Figure 7: JMeter HTTP Request samplers for all five CRUD operations and the HTTP Header Manager configuration.

Two listeners were used to capture and analyze the results of the test run: the View Results Tree, which displays each individual sample and its outcome, and the Summary Report, which aggregates statistics across all samples.

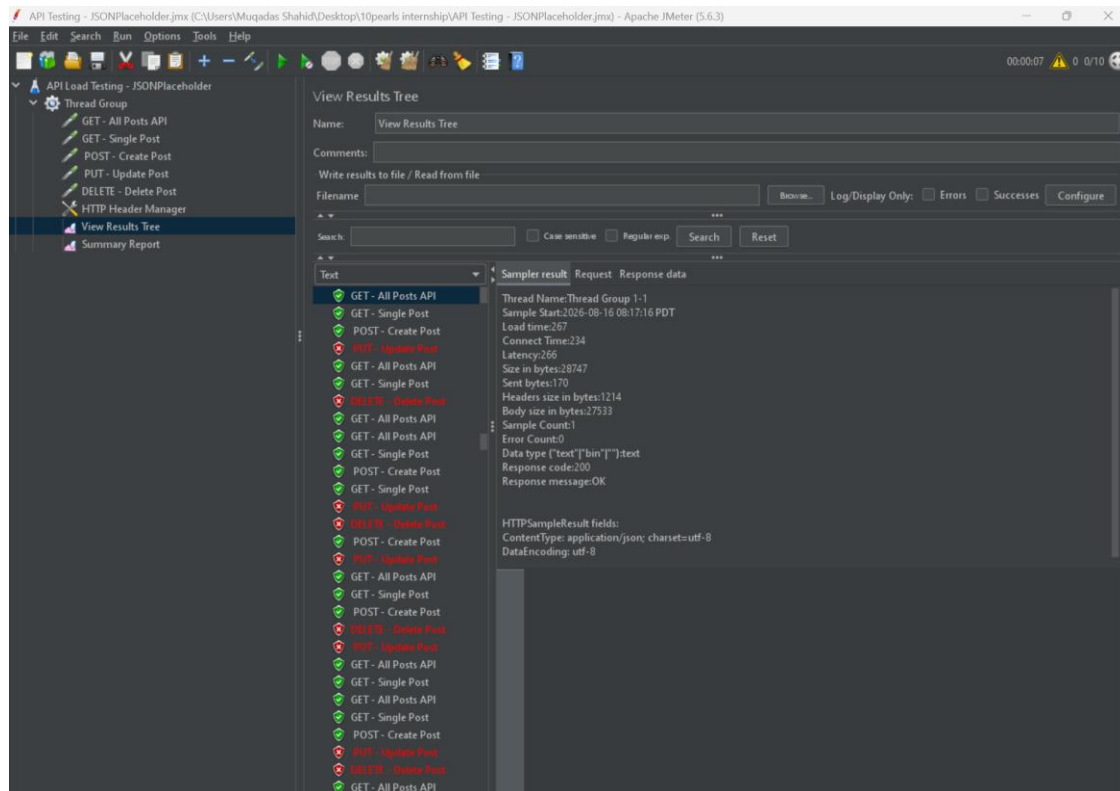


Figure 8: JMeter View Results Tree. Green entries indicate successful GET and POST requests; red entries indicate failed PUT and DELETE requests.

As shown in Figure 8, the GET - All Posts API and POST - Create Post requests completed successfully (green), while a large number of PUT - Update Post and DELETE - Delete Post requests are marked as failed (red) during the load test.

8. Performance Results and Analysis

The Summary Report listener produced the following aggregated results across the 50 samples executed for each request type:

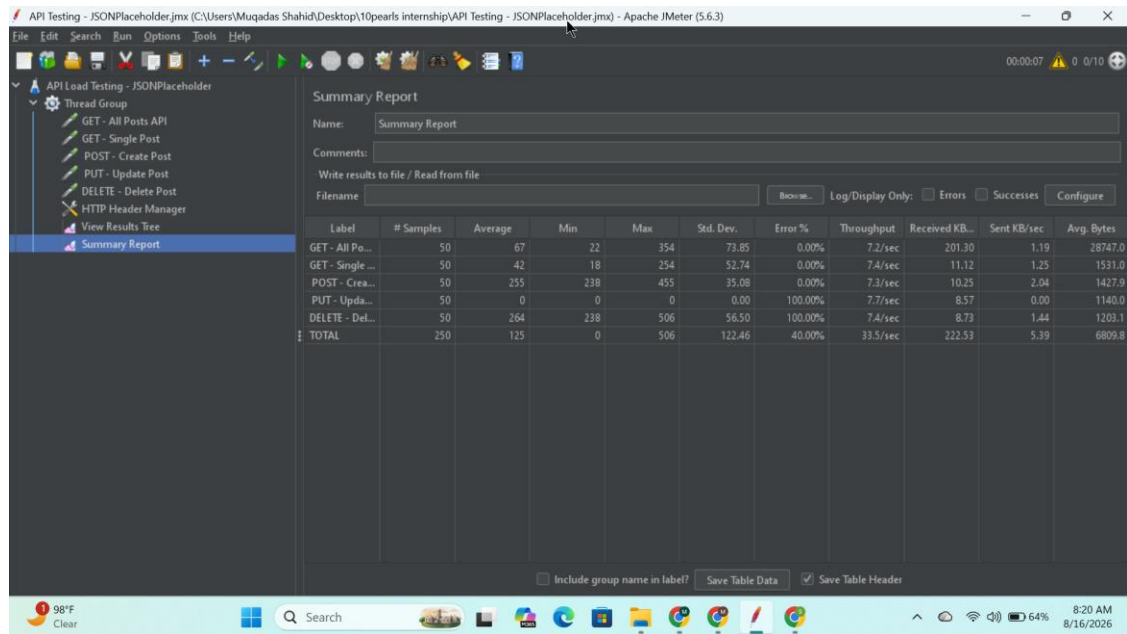


Figure 9: JMeter Summary Report showing response-time, throughput, and error-rate metrics for each request.

API Request	Samples	Average (ms)	Min (ms)	Max (ms)	Error %	Throughput
GET - All Posts	50	67	22	354	0.00%	7.2/sec
GET - Single Post	50	42	18	254	0.00%	7.4/sec
POST - Create Post	50	255	238	455	0.00%	7.3/sec
PUT - Update Post	50	0	0	0	100.00%	7.7/sec
DELETE - Delete Post	50	264	238	506	100.00%	7.4/sec
TOTAL	250	125	0	506	40.00%	33.5/sec

The GET - All Posts, GET - Single Post, and POST - Create Post requests completed with a 0.00% error rate across all 50 samples, with average response times of 67 ms, 42 ms, and 255 ms respectively. These results indicate that the API responded reliably and within acceptable time limits for read and create operations under the applied load of 10 concurrent users.

In contrast, the PUT - Update Post and DELETE - Delete Post requests recorded a 100.00% error rate across all 50 samples during the JMeter run, as reflected in both the Summary Report (Figure 9) and the View Results Tree (Figure 8). This is notable because the same PUT and DELETE operations were validated successfully in Postman (Figures 1–5), which indicates the failures are specific to the JMeter execution rather than the API design itself. Since the exact HTTP error returned by JMeter for these samplers was not captured in the provided screenshots, the precise root cause (for example, a request configuration or header/body mismatch under JMeter) is not confirmed here and would require further investigation using the Response Data tab in the View Results Tree.

The overall throughput across all request types combined was 33.5 requests per second, with a combined average response time of 125 ms and a total error rate of 40.00% (250 total samples, of which 100 — all PUT and DELETE samples — failed).

9. Key Findings

- All five CRUD operations (GET, GET, POST, PUT, DELETE) were tested against the JSONPlaceholder /posts endpoint.
- API responses were validated using automated Postman assertions covering status codes, JSON format, structure, and field-level data.
- The GET All Posts request successfully passed all 9 defined functional tests (9/9 PASSED).
- The Postman Collection Runner confirmed 35 of 35 automated tests passed across all five requests, with 0 failures and 0 errors.
- API performance was evaluated using Apache JMeter with 10 virtual users, a 5-second ramp-up period, and 5 loop iterations (50 samples per request).
- GET - All Posts, GET - Single Post, and POST - Create Post requests recorded a 0.00% error rate under load, with average response times between 42 ms and 255 ms.
- PUT - Update Post and DELETE - Delete Post requests recorded a 100.00% error rate during the JMeter load test, despite passing functional validation in Postman — a discrepancy that warrants further investigation.
- Performance metrics were analyzed using the View Results Tree and Summary Report listeners in JMeter.

10. Conclusion

This assignment provided practical experience in both API functional testing and API performance testing using industry-standard tools. Functional testing in Postman confirmed that all five CRUD operations on the JSONPlaceholder /posts endpoint behaved correctly, returning the expected status codes and JSON structures, with all 35 automated assertions passing successfully.

API quality assurance involves more than confirming that an endpoint returns a response. It requires validating status codes, response structure, and data integrity; automating functional assertions; and evaluating performance and error behavior under repeated or concurrent requests. The JMeter performance test in this assignment reinforced this point: while GET and POST requests performed reliably under load, the PUT and DELETE requests exhibited a 100% error rate, showing that functional correctness observed in Postman does not automatically guarantee identical behavior under a different testing tool or load condition, and that both types of testing are necessary for a complete quality assessment.

11. Learning Outcomes

- Practical, hands-on experience using Postman for API testing.

- Understanding of REST API CRUD operations (GET, POST, PUT, DELETE).
- Writing automated API assertions using Postman's pm.test() scripting API.
- Using the Postman Collection Runner to execute and consolidate multiple API test cases.
- Understanding of HTTP status codes and JSON response validation.
- Hands-on experience configuring and running performance tests with Apache JMeter.
- Understanding of virtual users (threads), ramp-up period, and loop count in load testing.
- Analyzing response time, throughput, and error rate from JMeter's View Results Tree and Summary Report.

12. GitHub Repository

GitHub Repository: <https://github.com/muqadasshahid-dev/Assignment-4-API-Testing>